



Database systems

ITCS2322

Chapter 11 : Database Performance Tuning and Query Optimization

اعداد : دكتور محمد عبد اللطيف راضي

Learning Objectives

- In this chapter, the student will learn:
 - Basic database performance-tuning concepts
 - How a DBMS processes SQL queries
 - About the importance of indexes in query processing
 - About the types of decisions the query optimizer has to make
 - Some common practices used to write efficient SQL code
 - How to formulate queries and tune the DBMS for optimal performance

Database Performance-Tuning Concepts

- End users interact with the DBMS through the use of queries to generate information, using the following sequence:
 - The end-user (client-end) application generates a query.
 - The query is sent to the DBMS (server end).
 - The DBMS (server end) executes the query.
 - The DBMS sends the resulting data set to the end-user (client-end) application.
- Goal of database performance is to execute queries as fast as possible

Database Performance-Tuning Concepts

- Database performance tuning: Set of activities and procedures that reduce response time of database system.
- The time required by a query to return a result set depends on many factors.
 - CPU,
 - (RAM), and
 - input/output (hard disk and network)

Table 11.1 - General Guidelines for Better System Performance

TABLE 11.1

GENERAL GUIDELINES FOR BETTER SYSTEM PERFORMANCE

	SYSTEM RESOURCES	CLIENT	SERVER
Hardware	CPU	The fastest possible Dual-core CPU or higher "Virtualized Client desktop technologies could also be used."	The fastest possible Multiple processors (quad-core technology or higher) Cluster of networked computers "Virtualized server technology could be used"
	RAM	The maximum possible to avoid OS memory to disk swapping	The maximum possible to avoid OS memory to disk swapping
	Storage	Fast SATA/EIDE hard disk with sufficient free hard disk space Solid state drives (SSDs) for faster speed	Multiple high-speed, high-capacity disks Fast disk interface (SAS / SCSI / Firewire / Fibre Channel) RAID configuration optimized for throughput Solid state drives (SSDs) for faster speed Separate disks for OS, DBMS, and data spaces
	Network	High-speed connection	High-speed connection
Software	Operating system (OS)	64-bit OS for larger address spaces Fine-tuned for best client application performance	64-bit OS for larger address spaces Fine-tuned for best server application performance
	Network	Fine-tuned for best throughput	Fine-tuned for best throughput
	Application	Optimize SQL in client application	Optimize DBMS server for best performance

Database Performance-Tuning Concepts

- Fine-tuning the performance of a system requires that all factors must operate at optimum level with minimal bottlenecks.
- Good database performance starts with good database design.
- No amount of fine tuning will make a poorly designed database perform as well as a well-designed database

Performance Tuning: Client and Server

- Client side
 - SQL performance tuning: Generates SQL query that returns correct answer in least amount of time
 - Using minimum amount of resources at server
- Server side
 - DBMS performance tuning: DBMS environment configured to respond to clients' requests as fast as possible
 - Optimum use of existing resources

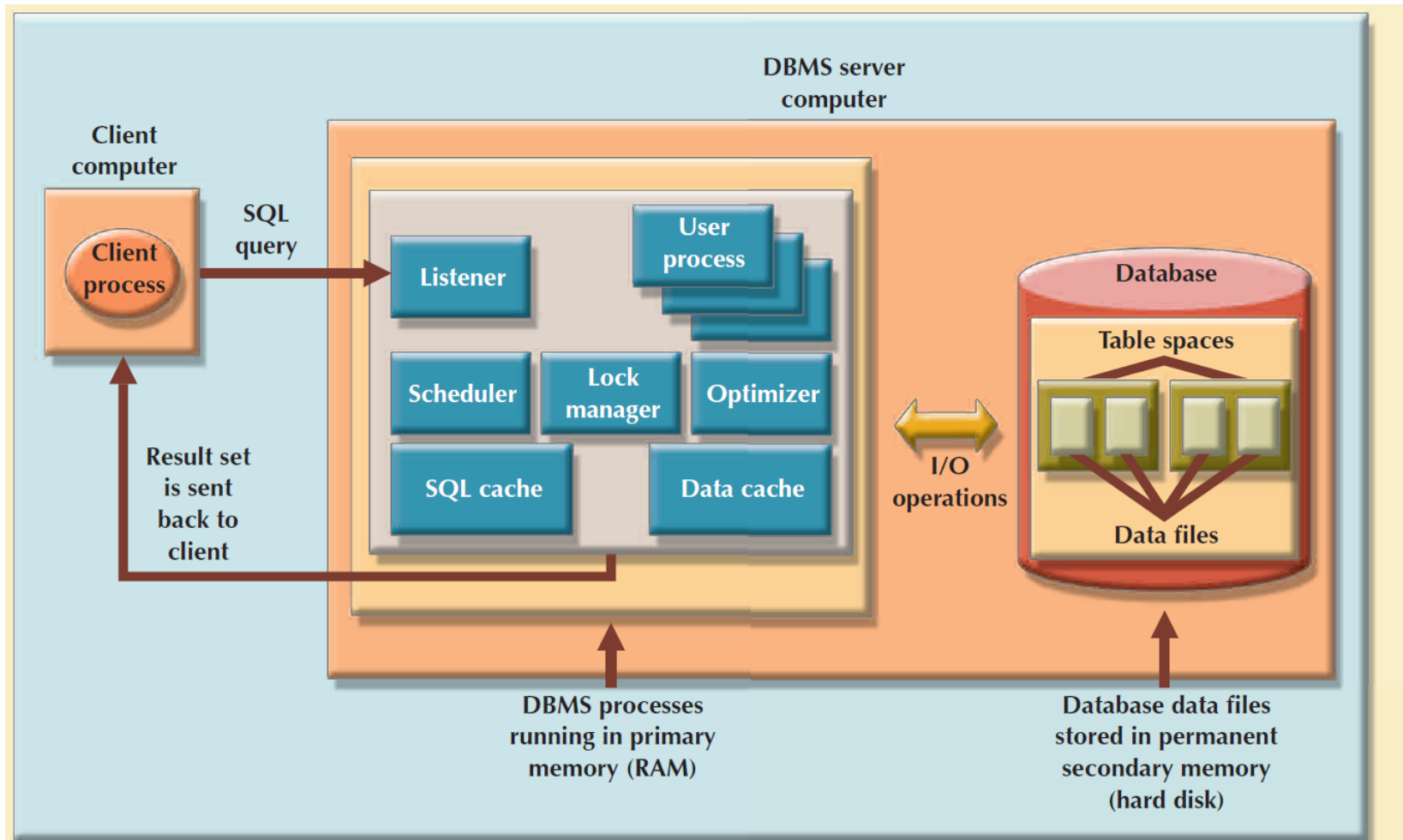
DBMS Architecture

- All data in a database are stored in data files
 - Data files automatically expand in predefined increments known as extends
- Data files are grouped in file groups or table spaces
 - Table space or file group: Logical grouping of several data files that store data with similar characteristics
- Data cache or buffer cache: Shared, reserved memory area
 - Stores most recently accessed data blocks in RAM

DBMS Architecture

- SQL cache or procedure cache: Stores most recently executed SQL statements or PL/SQL procedures
- DBMS retrieves data from permanent storage and places them in RAM
- Input/output request: Low-level data access operation that reads or writes data to and from computer devices
- Data cache is faster than working with data files
- Majority of performance-tuning activities focus on minimizing I/O operations

Figure 11.1 - Basic DBMS Architecture



Database Query Optimization Modes

- Algorithms proposed for query optimization are based on:
 - Selection of the optimum order to achieve the fastest execution time
 - Selection of sites to be accessed to minimize communication costs
- Evaluated on the basis of:
 - Operation mode
 - Timing of its optimization

Classification of Operation Modes

- Automatic query optimization: DBMS finds the most cost-effective access path without user intervention
- Manual query optimization: Requires that the optimization be selected and scheduled by the end user or programmer

Classification Based on Timing of Optimization

- Static query optimization: best optimization strategy is selected when the query is compiled by the DBMS
 - Takes place at compilation time
- Dynamic query optimization: Access strategy is dynamically determined by the DBMS at run time, using the most up-to-date information about the database
 - Takes place at execution time

Classification Based on Type of Information Used to Optimize the Query

- Statistically based query optimization algorithm: Statistics are used by the DBMS to determine the best access strategy
- Statistical information is generated by DBMS through:
 - Dynamic statistical generation mode
 - Manual statistical generation mode
- Rule-based query optimization algorithm: based on a set of user-defined rules to determine the best query access strategy

Table 11.2 - Sample Database Statistics Measurements

TABLE 11.2

SAMPLE DATABASE STATISTICS MEASUREMENTS

DATABASE OBJECT	SAMPLE MEASUREMENTS
Tables	Number of rows, number of disk blocks used, row length, number of columns in each row, number of distinct values in each column, maximum value in each column, minimum value in each column, and columns that have indexes
Indexes	Number and name of columns in the index key, number of key values in the index, number of distinct key values in the index key, histogram of key values in an index, and number of disk pages used by the index
Environment Resources	Logical and physical disk block size, location and size of data files, and number of extents per data file

Query Processing

Parsing

- DBMS parses the SQL query and chooses the most efficient access/execution plan

Execution

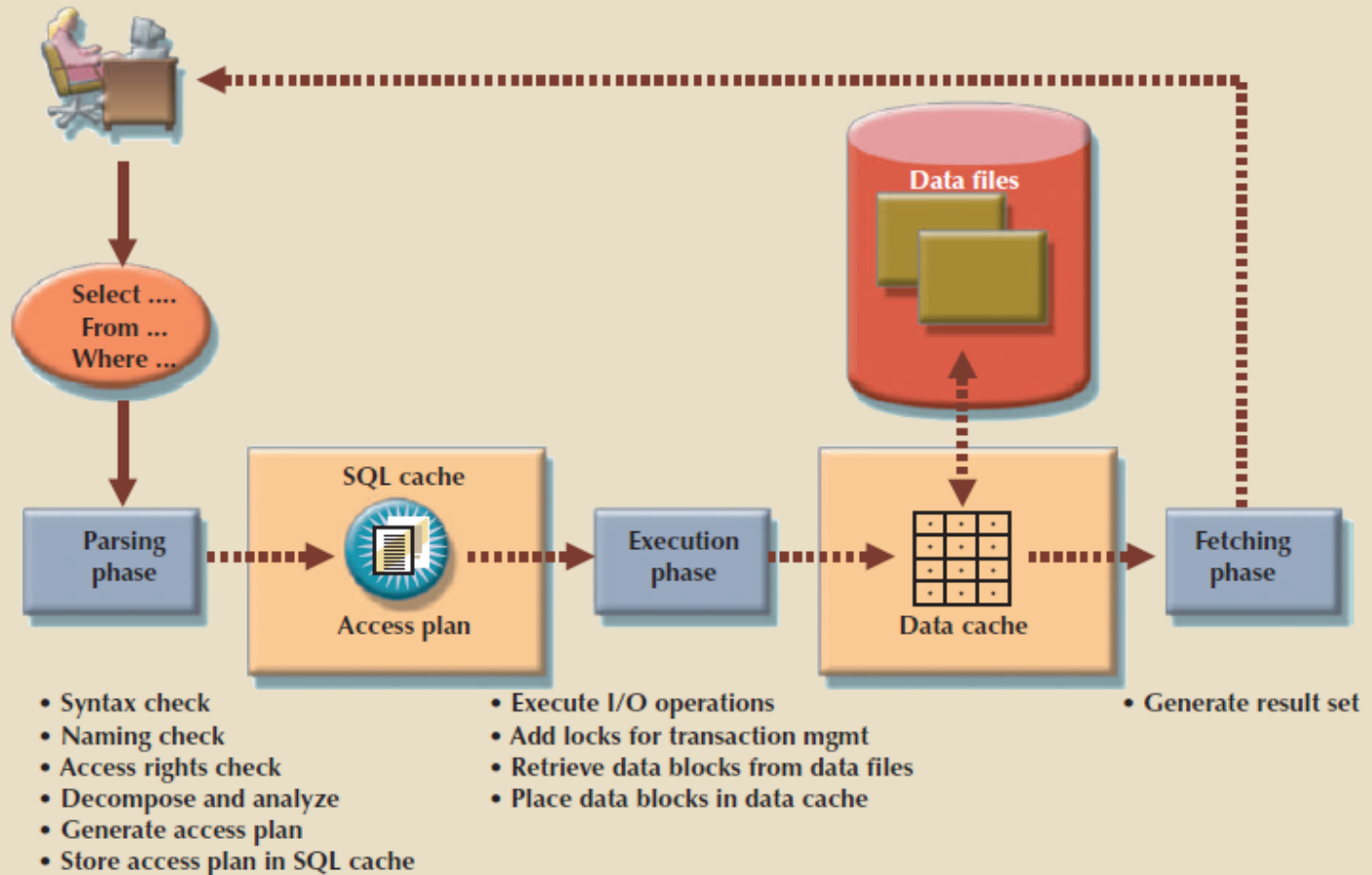
- DBMS executes the SQL query using the chosen execution plan

Fetching

- DBMS fetches the data and sends the result set back to the client

Query Processing

FIGURE 11.2 QUERY PROCESSING



SQL Parsing Phase

- Query is broken down into smaller units
- Original SQL query is transformed into slightly different version of the original SQL code which is fully equivalent and more efficient
- Query optimizer: Analyzes SQL query and finds most efficient way to access data
- Access plans: DBMS-specific and translate client's SQL query into a series of complex I/O operations

SQL Parsing Phase

- If access plan already exists for query in SQL cache, DBMS reuses it
- If not, optimizer evaluates various plans and chooses one to be placed in SQL cache for use

SQL Parsing Phase

- Parsing a SQL query requires several steps:
- Validated for syntax compliance
- Validated against the data dictionary to ensure that table names and column names are correct.
- Validated against the data dictionary to ensure that the user has proper access rights
- Analyzed and decomposed into more atomic components
- Optimized through transformation into a fully equivalent but more efficient SQL query
- Prepared for execution by determining the most efficient execution or access plan

SQL Execution Phase

- All I/O operations indicated in the access plan are executed
 - Locks are acquired
 - Data are retrieved and placed in data cache
 - Transaction management commands are processed

SQL Fetching Phase

- Rows of resulting query result set are returned to client
- DBMS may use temporary table space to store temporary data
- Database server coordinates the movement of the result set rows from the server cache to the client cache

Query Processing Bottlenecks

- Delay introduced in the processing of an I/O operation that slows the system
- Caused by the:
 - CPU
 - RAM
 - Hard disk
 - Network
 - Application code

Indexes and Query Optimization

- Indexes
 - Help speed up data access
 - Facilitate searching, sorting, using aggregate functions, and join operations
 - Ordered set of values that contain the index key and pointers
 - More efficient than a full table scan

Indexes and Query Optimization

FIGURE 11.3 INDEX REPRESENTATION FOR THE CUSTOMER TABLE



Indexes and Query Optimization

- One measure that determines the need for an index is the data sparsity of the column you want to index.
- Data sparsity: Number of different values a column could have.
- Knowing the sparsity helps you decide whether the use of an index is appropriate
 - column with high sparsity: index processing might be necessary work
 - column with low sparsity: index processing might be unnecessary work.

Indexes and Query Optimization

- Data structures used to implement indexes:
 - Hash indexes
 - B-tree indexes
 - Bitmap indexes
- DBMSs determine best type of index to use

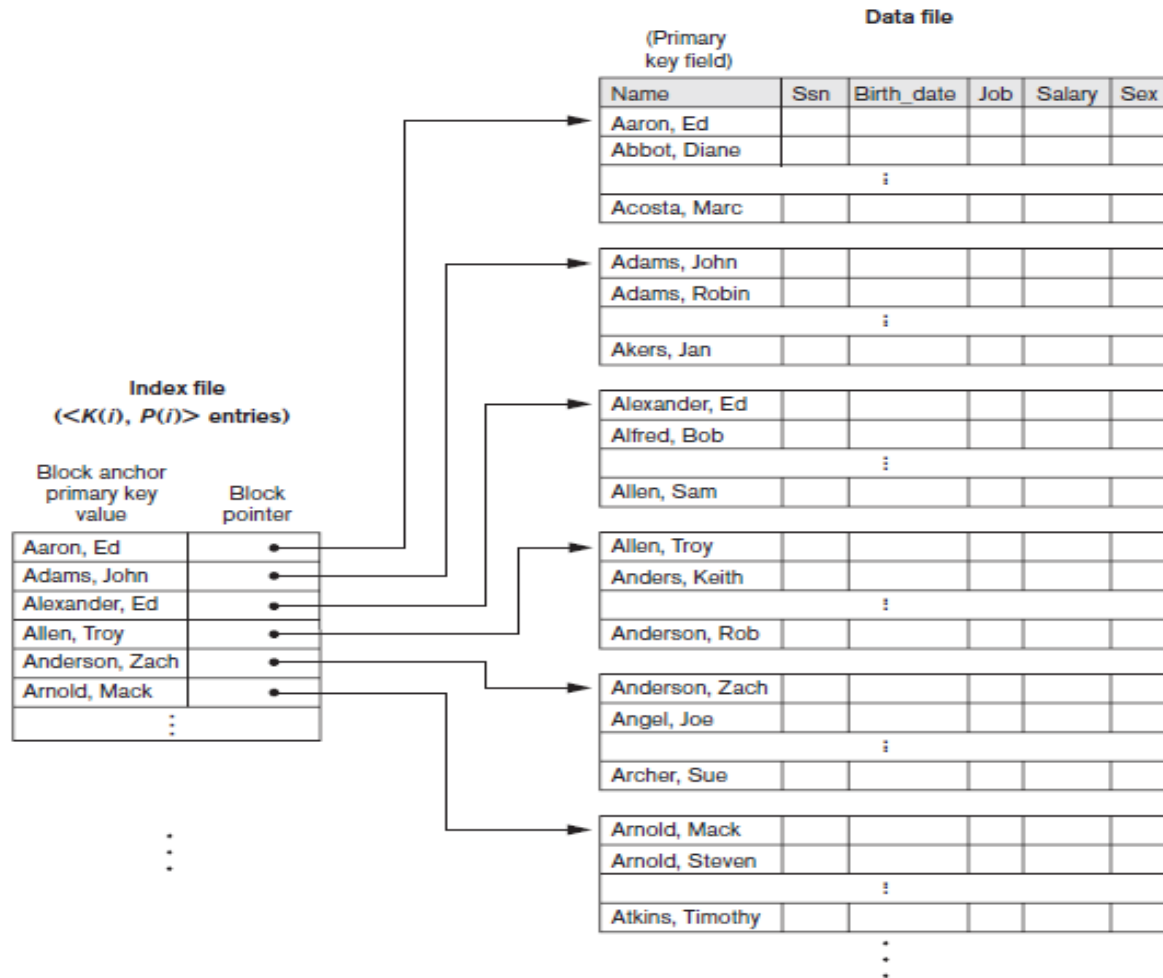
Indexes and Query Optimization

- **Hash index.** A hash index is based on an ordered list of hash values.
- A hash algorithm is used to create a hash value from a key column.
- This value points to an entry in a hash table, which in turn points to the actual location of the data row.
- **This type of index is good for simple and fast lookup operations based on equality conditions.**
- For example, LNAME="Scott" and FNAME="Shannon".

Primary Indexes

- Ordered file with two fields
 - Primary key, $K(i)$
 - Pointer to a disk block, $P(i)$
- One index entry in the index file for each block in the data file
- Indexes may be dense or sparse
 - Dense index has an index entry for every search key value in the data file
 - Sparse index has entries for only some search values

Primary Indexes



Primary Indexes

- Major problem: insertion and deletion of records
- Move records around and change index values
- Solutions
 - Use unordered overflow file
 - Use linked list of overflow records

Indexes and Query Optimization

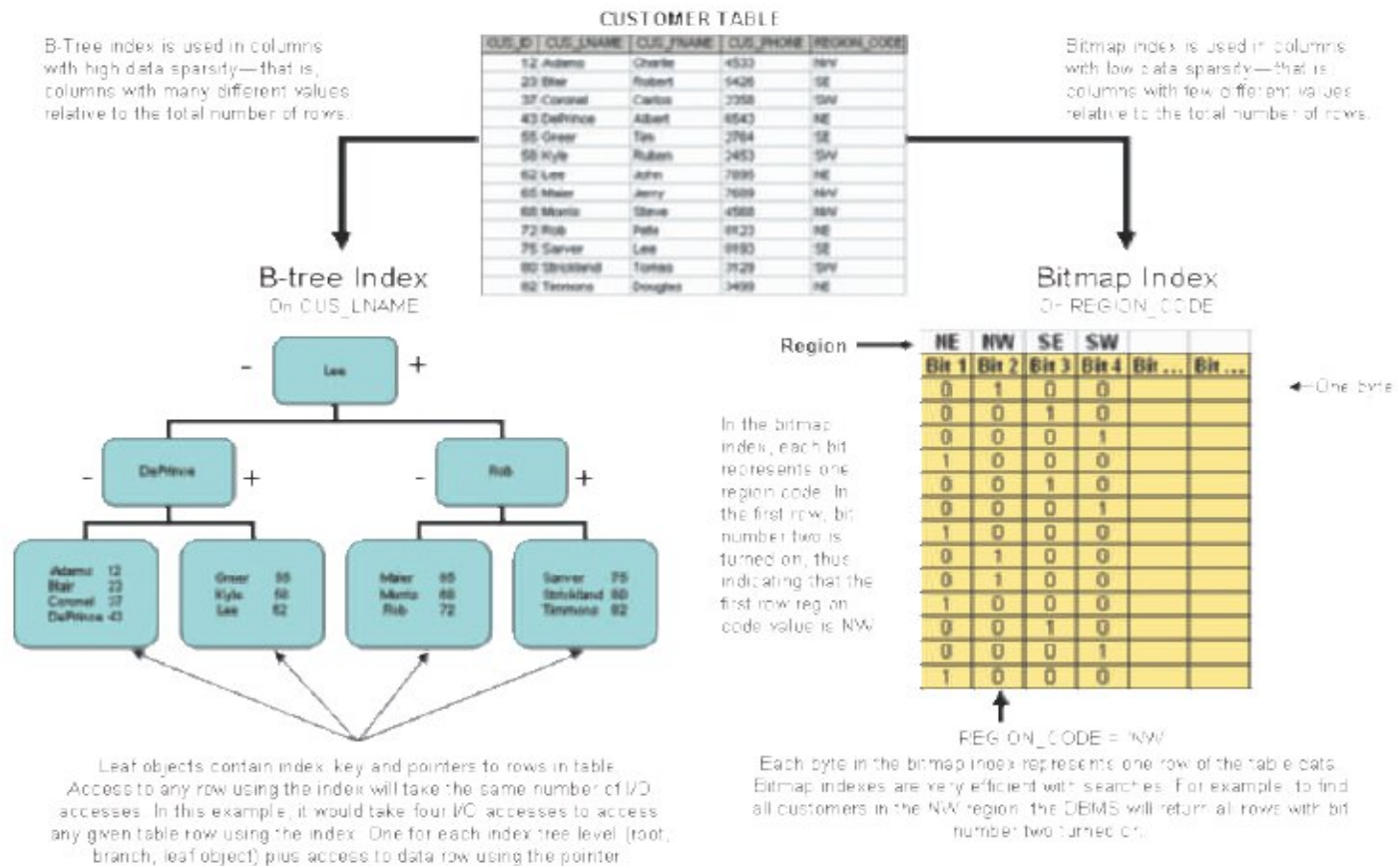
- **The B-tree index** is an ordered data structure organized as an up sided own tree.
- The index tree is stored separately from the data.
- The lower-level leaves of the B-tree index contain the pointers to the actual data rows.
- B-tree indexes are “self-balanced,” which means that it takes approximately the same amount of time to access any given row in the index.
- This is the default and most common type of index used in databases.
- **The B-tree index is used mainly in tables in which column values repeat a relatively small number of times.**

Indexes and Query Optimization

- A **bitmap index** uses a bit array (0s and 1s) to represent the existence of a value or condition.
- These indexes are used mostly in data warehouse applications in tables with a large number of rows in which a small number of column values repeat many times.
- Bitmap indexes tend to use less space than B-tree indexes because they use bits instead of bytes to store their data.

Indexes and Query Optimization

FIGURE 11.4 B-TREE AND BITMAP INDEX REPRESENTATION



Optimizer Choices

- DBMS must choose what indexes to use, how to perform join operations, which table to use first, and so on.
- The query optimizer can operate in one of two modes:
 - Rule-based optimizer: Uses preset rules and points to determine the best approach to execute a query
 - Cost-based optimizer: Uses algorithms based on statistics about objects being accessed to determine the best approach to execute a query

Using Hints to Affect Optimizer Choices

- Optimizer might not choose the best execution plan
 - Makes decisions based on existing statistics, which might be old
 - Might choose less-efficient decisions
- Optimizer hints: Special instructions for the optimizer, embedded in the SQL command text

Table 11.5 - Optimizer Hints

TABLE 11.5

OPTIMIZER HINTS

HINT	USAGE
ALL_ROWS	Instructs the optimizer to minimize the overall execution time—that is, to minimize the time needed to return all rows in the query result set. This hint is generally used for batch mode processes. For example: <pre>SELECT /*+ ALL_ROWS */ * FROM PRODUCT WHERE P_QOH < 10;</pre>
FIRST_ROWS	Instructs the optimizer to minimize the time needed to process the first set of rows—that is, to minimize the time needed to return only the first set of rows in the query result set. This hint is generally used for interactive mode processes. For example: <pre>SELECT /*+ FIRST_ROWS */ * FROM PRODUCT WHERE P_QOH < 10;</pre>
INDEX(name)	Forces the optimizer to use the P_QOH_NDX index to process this query. For example: <pre>SELECT /*+ INDEX(P_QOH_NDX) */ * FROM PRODUCT WHERE P_QOH < 10</pre>

SQL Performance Tuning

- Evaluated from client perspective
 - Most current relational DBMSs perform automatic query optimization at the server end
 - Most SQL performance optimization techniques are DBMS-specific and thus rarely portable
- Majority of performance problems are related to poorly written SQL code

Index Selectivity

- Measure of the likelihood that an index will be used in query processing
- Indexes are used when a subset of rows from a large table is to be selected based on a given condition
- Index cannot always be used to improve performance
- Function-based index: Based on a specific SQL function or expression

Conditional Expressions

- Expressed within WHERE or HAVING clauses of a SQL statement
- Restricts the output of a query to only rows matching conditional criteria
- Guidelines to write efficient conditional expressions in SQL code
 - Use simple columns or literals as operands
 - Numeric field comparisons are faster than character, date, and NULL comparisons

Conditional Expressions

- Equality comparisons are faster than inequality comparisons
- Transform conditional expressions to use literals
- Write equality conditions first when using multiple conditional expressions
- When using multiple AND conditions, write the condition most likely to be false first
- When using multiple OR conditions, put the condition most likely to be true first
- Avoid the use of NOT logical operator

Query Formulation

- Identify what columns and computations are required
- Identify source tables
- Determine how to join tables
- Determine what selection criteria are needed
- Determine the order in which to display the output

DBMS Performance Tuning

- Managing DBMS processes in primary memory and the structures in physical storage
- DBMS performance tuning at server end focuses on setting parameters used for:
 - Data cache
 - SQL cache
 - Sort cache
 - Optimizer mode
- In-memory database: Store large portions of the database in primary storage

DBMS Performance Tuning

- Recommendations for physical storage of databases:
 - Use RAID (Redundant Array of Independent Disks) to provide a balance between performance improvement and fault tolerance
 - Minimize disk contention
 - Put high-usage tables in their own table spaces
 - Assign separate data files in separate storage volumes for indexes, system, and high-usage tables

DBMS Performance Tuning

- Take advantage of the various table storage organizations in the database
 - Index-organized table or clustered index table: Stores the end-user data and the index data in consecutive locations in permanent storage
- Partition tables based on usage
- Use denormalized tables where appropriate
- Store computed and aggregate attributes in tables